

LanFileShare

局域网文件快传 · 一扫码就把手机照片传到电脑

版本 1.0.5 · 2026-08 · C# + WPF · .NET 8

目录

项目概览	文件上传与归档
技术选型	失败容错
系统架构	打包与分发
主程序界面设计	配套脚本
手机端界面设计	所有设计决策一览
端口方案	验收场景
HTTP 服务器设计	版本与变更记录

1 项目概览

◆ 目标

做一个 **Windows 单文件 EXE**，打开后主界面显示一个二维码，同一局域网内的手机扫码即可把**图片或文档**上传到电脑指定文件夹。

◆ 核心诉求

- 扫码即用（无需注册、无需安装 App）
- 完全离线运行（不连外网）
- 不限制文件类型（手机端可推任何文件）
- 同时多文件并发上传
- 自动按手机、日期归档
- 关闭后仍在后台运行（手机继续传）

◆ 典型场景

- 家里拍了十几张照片想快速传到电脑
- 办公场景：同事手机扫码传 docx 到他的电脑
- 外出录的视频回酒店后想批量传到笔记本

◆ 交付物

入口

解压即用的单 EXE

体积

~10 MB（含 .NET 运行时）

依赖

零运行时依赖

分发

ZIP 包，单击 EXE 即用

2 技术选型

◆ 核心决策

维度	选择	理由
语言 + 框架	C# + WPF + .NET 8	Windows 原生、自包含 EXE (5–10 MB)、并发能力强、杀软友好
二维码	<code>QRCode</code> 1.6.0	NuGet 上最稳定的二维码库，输出 PNG/Bitmap
系统托盘	<code>Hardcodet.NotifyIcon.Wpf</code> 2.0.1	纯 WPF 实现，规避与 WinForms 共存的命名空间冲突
HTTP 服务	自己写的 TcpListener HTTP/1.1	绕过 <code>HttpListener</code> / <code>http.sys</code> 在部分机器的 silent failure
端口检测	Raw Socket (<code>IPAddress.Any</code>) 试绑	真和系统服务抢端口，比 <code>TcpListener(127.0.0.1)</code> 可靠
打包	<code>dotnet publish --self-contained</code>	单文件 + 嵌入运行时 + 压缩

◆ 为什么不用 HttpListener / Kestrel

框架	问题
HttpListener	在某些 Windows 上 <code>Start()</code> 对占用端口 静默成功 (PID 4 System 占同端口时)，实际 TCP 包路由给 System。可靠性差。

框架	问题
Kestrel	需要 <code>Microsoft.AspNetCore.App</code> shared framework, EXE 体积 +30MB, 对一个桌面工具过重
Self-hosted (采纳)	<code>TcpListener</code> + 自己解析 <code>HTTP/1.1 + multipart</code> , 零额外依赖, 体积 0 增长

◆ 依赖清单

```
<PackageReference Include="QRCoder" Version="1.6.0" />
<PackageReference Include="Hardcodet.NotifyIcon.Wpf" Version="2.0.1" />
<!-- 其余 .NET 8 内置: HttpClient / System.IO / System.Text.Json / Task / TcpListener -->
```

3 系统架构

◆ 模块划分

```
LanFileShare/
├─ App.xaml(.cs)                # 入口、捕获全局异常
├─ MainWindow.xaml(.cs)         # 主窗口: QR + 路径 + 托盘
├─ Models/
│   ├─ AppSettings.cs           # 配置 DTO
│   └─ AllowedExtensions.cs     # 文件类型白名单 (备用)
├─ Services/
│   ├─ SettingsStore.cs         # JSON 持久化 (%APPDATA%\LanFileShare\Settings.json)
│   ├─ Logger.cs                # 文件 + 7天滚动日志
│   ├─ PortSelector.cs          # 自动选空闲端口 (50000-50999)
│   ├─ PortProbe.cs             # raw socket + IPAddress.Any 真部检测
│   ├─ QrGenerator.cs           # QRCoder 包装
│   ├─ TrayIcon.cs              # Hardcodet TaskbarIcon
│   ├─ TcpHttpServer.cs         # 用户态 HTTP/1.1 服务器 (核心)
│   ├─ FileReceiver.cs          # multipart 解析 + 写盘
│   └─ IpDiscovery.cs           # 选局域网 IPv4
└─ Helpers/
```

```

├── MultipartParser.cs      # multipart 解析 (旧版保留)
├── Resources/
│   ├── appicon.png        # 应用图标 (嵌入程序集)
│   ├── MobileHtml.cs      # mobile.html 内联字符串
│   └── LanFileShare.csproj

```

◆ 数据流（一次上传）

☑ 手机浏览器

🖥 PC 端 LanFileShare

```

1. 扫码 → 加载 mobile.html  ← GET / ← TcpHttpServer
                               ↓ (HTML 1280+ 字符)
                               缓存到 MobileHtml.Content
                               |
                               | 选文件 (FormData) |
                               |
2. 点 "上传" (XHR)           → POST /upload → HandleUploadAsync
                               body = multipart (header + body)
                               Content-Type: multipart/form-data;
                                                             boundary=---Web
                               Content-Length: 12345 (1.5MB)
                               |
                               FileReceiver.HandleUploadAsync
                               → 读完整 body (200MB MAX)
                               → ExtractBoundary(contentType)
                               → ParseAndSave 写每个 part
                               ↓
                               {success: true, saved: 1}
onload
3. ✓ 卡片翻转"已完成"

```

4 主程序界面设计

◆ 窗口规格

- 尺寸：420 × 560，不可拉伸（`ResizeMode="CanMinimize"`）
- 居中显示，关闭按钮收起到系统托盘

◆ 布局（从顶到底）



◆ 图标设计

- 主窗口 `Icon="/Resources/appicon.png"`
- 系统托盘: `TrayIcon.BuildAppIcon()` 从嵌入 PNG 转 `System.Drawing.Icon`
- 图标: 蓝色渐变背景 + 白色 QR finder pattern + 绿色上传箭头

5 手机端界面设计

◆ 整体风格

- **蓝色 hero 区**：渐变背景（#2563EB → #1D4ED8）+ 白图标 + 标题 + "用户N · 已连接" 状态条（带绿色心跳点）
- **卡片化**：内容用白色圆角卡片 + 阴影呈现
- **扁平现代**：接近 iOS / 微信小程序的观感

◆ 三层布局

层	内容
Hero header	大图标 + "发送文件到电脑" 标题 + 设备名状态
中间区	Drop zone / 文件列表 / 全局进度条 / 上传按钮 / 完成态
Toast（fixed）	失败时浮出红色 toast，3 秒自动收起

◆ 完成态按结果区分（容错）

上传结果	显示内容
全部成功	大对勾✓ "已成功上传 N 个文件" + "继续上传" 按钮
部分失败	警告 ⚠ "成功 X · 失败 Y" + "重试失败项" 按钮（ 不显示"上传完成"字样 ）
全部失败	完全不显示完成页 ，留在原页面，列表里逐个文件显示具体错误 + 顶部 toast

◆ 设备标识逻辑

浏览器 `localStorage` 持久化设备名：

- 首次访问 → 读 `lastUserNum` (默认 0) +1 → 存为 `deviceName` = "用户N"
- 再次访问 → 沿用之前的 `deviceName`
- 新浏览器 + 新设备 → 自动递增 (用户1, 用户2, ...)
- Android/iOS 浏览器清理数据后访问 → 自动拿到新编号

6 端口方案

◆ 范围设定

默认 50000-50999 (1000 个端口)。

低端口 (0-30000) 容易被各种 Windows 系统服务、Hyper-V 端口映射、WinNAT 等 "PID 4 (System)" 偷偷占着。我们机器实测连 10000-10999、50000-50999 都有 System 在 LISTEN, 看似没人用但 TCP 包路由给 System。50000+ 仍占则用 raw socket 试 0.0.0.0 直接对抗。

◆ 候选顺序

```
for p in [lastPort, lastPort+1, lastPort-1, lastPort+2, lastPort-2, ...]:
    if PortProbe.IsFree(p):
        # Socket.Bind(IPAddress.Any, p)
        return p
    continue
return -1
# 区间被占满 (理论不会)
```

◆ 探测方式（关键）

```
// PortProbe.IsFree: 裸 socket + IPAddress.Any
private static bool IsFree(int port)
{
    var sock = new Socket(AddressFamily.InterNetwork, SocketType.Stream, ProtocolType.Tcp);
    try
    {
        sock.Bind(new IPEndPoint(IPAddress.Any, port)); // 真和 System 抢
        return true;
    }
    catch (SocketException) { return false; }
    finally { sock?.Close(); }
}
```

用 `0.0.0.0` 而非 `127.0.0.1` 探测：能检测 System 是否在抢这个端口（System bind 时只在 127.0.0.1 试可能成功但被允许，会骗过旧版）。

7 HTTP 服务器设计

◆ 设计目标

- 完全跳过 `HttpListener / http.sys`
- 在用户态 socket 上直接服务 HTTP/1.1
- 支持 `multipart/form-data`（浏览器 `FormData` 上传）

◆ 核心服务（`TcpHttpServer.cs`）

```
public class TcpHttpServer
{
    private readonly TcpListener _listener;
    private CancellationTokenSource _cts;

    public static TcpHttpServer? TryStart(int port, SettingsStore s, Action onDone)
    {
        if (!PortProbe.IsFree(port)) return null; // 第一道闸
```

```

        var listener = new TcpListener(IPAddress.Any, port);
        listener.Start();
        Logger.Info($"port {port} bind OK on 0.0.0.0");
        return new TcpHttpServer(listener, port, s, onDone);
    }

    private async Task AcceptLoopAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            var client = await _listener.AcceptTcpClientAsync(ct);
            _ = Task.Run(() => SafeHandle(client));
        }
    }

    // 每连接独立 Task，线程安全无共享状态
}

```

◆ 路由表

路径	方法	行为
/	GET	返回 mobile.html（移动端首页）
/test	GET	Hello from LanFileShare! Port=... （自检用）
/info	GET	返回 mobile.html 长度（自检用）
/health	GET	{success:true, port:...} JSON
/upload	POST	接收 multipart，写盘
/favicon.ico	GET	204 No Content（避免浏览器反复请求）
其他	*	404 Not Found

wechat-image.jpg
用户3/ ← 设备 C

◆ 目录命名规则

- 设备名: `SanitizeFolder(s)` 去掉 `\ / : * ? " < > |`, 限制 50 字符
- 日期: `DateTime.Now.ToString("yyyy-MM-dd")`
- 文件名: `SanitizeFileName(s)` 去掉 `Path.GetInvalidFileNameChars()`, 限制 200 字符

9 失败容错

◆ 三层防御

层	触发	处理
客户端 (mobile.html)	任意一个 <code>uploadOne</code> reject (HTTP 非 2xx、网络 错误、超时、解析失 败)	该 item.status = 'fail', 显 示具体错误, 重试按钮可用
客户端 (汇总)	全部 fail / 部分 fail	不显示"上传完成"标识; 部 分 fail 时给 "⚠ 部分上传完 成" + "重试失败项"
服务端 (tcp server)	multipart 解析异常、 保存失败	写 log, 返 4xx/5xx + JSON <code>{success:false, error:...}</code>
任何未捕获异常		

层	触发	处理
服务端 (catch-all)		switch 块 try/catch 兜底, 返 500 + log

◆ 错误响应协议 (mobile.html 用得到)

```
// 成功
{ "success": true, "saved": 3 }

// 业务失败
{ "success": false, "error": "文件类型不允许" }
{ "success": false, "error": "body too large: 209715200 > 209715200" }
{ "success": false, "error": "missing Content-Length" }

// HTTP 层失败
HTTP 500 { "success": false, "error": "server exception: IOException: ..." }
```

◆ 前端识别两个字段名 (兼容老版本)

```
if (resp.success || resp.ok) { resolve(resp); }
else { reject(new Error(resp.error || '...')) }
```

10 打包与分发

◆ 打包命令

```
dotnet publish -c Release -r win-x64 `
  --self-contained true `
  /p:PublishSingleFile=true `
  /p:IncludeNativeLibrariesForSelfExtract=true `
```

```
/p:EnableCompressionInSingleFile=true `
./LanFileShare/LanFileShare.csproj -o ./publish
```

◆ 产物规格

EXE	<code>publish/LanFileShare.exe</code> (~5-10 MB, 启用压缩)
首次启动	解压依赖到 <code>%LOCALAPPDATA%\Temp\.net\LanFileShare\</code> (1-2 秒)
后续启动	几乎零延迟
ZIP 打包	<code>dist/LanFileShare-v<Version>-win-x64.zip</code> (同样 ~5-10 MB)

◆ 为什么用 ZIP 而不是裸 EXE

很多即时通讯软件（QQ、企业微信）和邮件系统默认拦截 `.exe` 附件。ZIP 包通常可直接发送。ZIP 内只有 1 个 EXE，无 readme / 无 dll / 无 setup.ini，最大兼容各种分发渠道。

◆ 版本与变更

遵循 **SemVer 2.0** 约定：

- `v1.0.0` → MVP 上线
- `v1.0.5` → 修 bug（当前）
- `v1.1.0` → 加功能（如断点续传）
- `v2.0.0` → 大改（改架构时）

11 配套脚本

◆ build.ps1 / build.cmd

一键构建 + 错误日志：

- 杀掉残留进程（避免文件占用）
- 清理 build 产物
- 运行 `dotnet publish`
- 失败时把完整 log dump 到 `.build-logs/last-error.log` + 屏幕
- 成功后提示 EXE 时间戳（验证是真的新 EXE）

◆ setup.cmd

首次运行配置（需管理员，自动提权）：

- 添加防火墙入站规则（`LanFileShare HTTP (auto)`）
- 检测 .NET 8 Desktop Runtime
- 重置 `LastPort` 到新范围（避开旧值 9002 / 10000）
- HTTP 回环测试（健康检查）

◆ diagnose.cmd

任何时候的诊断报告（不需要管理员）：

- [A] .NET Runtime 列表
- [B] 本机 IPv4
- [C] 当前 LISTENING 端口（含 PID）
- [C2] 低端口 PID 4 陷阱
- [D] 防火墙规则详情
- [E] EXE 进程状态
- [F] 最后 20 行日志
- [G] AppData 目录

12 所有设计决策一览

#	决策点	最终选择
1	开发语言	C# + WPF + .NET 8
2	分发形式	单文件绿色 EXE（自包含，~10MB）
3	界面风格 (PC)	极简单页风
4	手机端风格	蓝色渐变 hero + 卡片化文件列表
5	HTTP 服务	自己写 TcpListener HTTP/1.1（绕开 HttpListener）
6	端口范围	50000-50999， LastPort 仅作偏好
7	端口探测	Raw Socket + IPAddress.Any（真和 System 抢）
8	安全验证	不设验证（局域网信任环境）
9	文件类型	不限制（浏览器原生 accept）
10	同名冲突	自动重命名 (1) (2)
11	文件大小	无限制（默认 200MB 上限，可调）
12	归档方式	按手机（用户N）+ 按日期
13	设备命名	计数器递增（用户1, 用户2, ...）
14	进度反馈	双向：手机端全局进度 + 每个文件进度；PC 端静默
15	PC 窗口行为	关闭→最小化到托盘

#	决策点	最终选择
16	并发手机	最多 ~10 台 (C# TcpListener 轻松应付)
17	PC 界面信息	极简: QR 码 + 路径选择器
18	路径记忆	记忆上次路径, 丢失提示
19	主程序图标	嵌入 PNG (标题栏 + 任务栏 + 托盘统一)
20	完成态容错	按结果区分 (详见 § 9)
21	失败展示	具体错误文本 + 重试按钮 (不显示"上传完成"标识)
22	首次运行	<code>setup.cmd</code> 自带防火墙规则

13 验收场景

◆ 功能验证

- ☒ 启动应用, 看到蓝色图标、二维码、保存路径选择器
- ☒ 手机扫码后加载蓝色 hero 区页面, 看到当前设备名
- ☒ 选择 1 张照片, 点击"开始上传"
- ☒ 文件出现在 `保存目录/用户1/2026-XX-XX/` 下
- ☒ 同时上传 5 张, 前 3 个并发, 进度条动
- ☒ 全部成功 → 显示大对勾"上传完成"

◆ 容错验证

- ☒ 关掉 PC 服务后再上传 → 手机端显示具体错误 + 不显示"上传完成"
- ☒ 保存路径改成不可写的位置 → 写入失败 → 文件显示"失败", 可重试

- ☒ 部分成功 → 显示"⚠ 部分完成" + 重试按钮
- ☒ 浏览器关闭 localStorage → 设备名重置为"用户1"

◆ 兼容性 / 边界

- ☒ 关闭窗口 → 应用退到托盘，仍可上传
- ☒ 重启应用 → 路径自动恢复
- ☒ 端口 50000 被占 → 自动跳到 50001
- ☒ 不同设备 → 不同子目录（用户1 / 用户2）
- ☒ 5+ 并发上传 → 浏览器性能 OK

◆ 环境兼容

- ☒ Win10 / Win11 双击 EXE 启动
- ☒ 端口被 System (PID 4) 占 → 自动换
- ☒ 防火墙阻止 → setup.cmd 自动添加规则

14 版本与变更记录

◆ v1.0.5（当前版本）

- ☒ 主程序界面：增加应用图标（标题栏、任务栏、托盘）
- ☒ 手机端页面：完全重写美化，蓝色 hero + 卡片化
- ♥ 完成态智能显示：成功/部分失败/全失败 三态分别处理
- ♥ 失败时显示具体错误 + 单个重试按钮
- ☒ HTTP 服务器：抛弃 HttpListener，用 raw TcpListener 实现
- ☒ 端口检测：用 raw socket + IPAddress.Any 真绑 0.0.0.0
- ☒ 端口范围：50000-50999（避开 9000-10999 的 System 占用区）
- ☒ build 脚本：失败时 dump 完整 log 到 `.build-logs/last-error.log`

- 修: X-Device-Id 中文编码问题 (`encodeURIComponent`)
- 修: JSON 字段不一致问题 (`ok` → `success`)
- 修: multipart boundary 从 Content-Type 取 (不是从 body 找)

◆ v1.0.0 (MVP)

- 初版发布, HttpListener 实现
- 端口固定 9000, 被 System 占导致连不上

 项目源码: LanFileShare-v1.0.0-source.zip

 文档版本: v1.0.5 (2026-08)

[↑ 返回顶部](#)